Java Factory Pattern - Cheat Sheet

Factory is the “let someone else decide which class to instantiate” pattern. The caller asks for an object by description (a type string, an enum, a config flag) and gets back the right concrete class without ever calling `new` themselves.

Useful when:

- The exact class depends on runtime info you don’t have at compile time.
- The construction logic is gnarly (config files, network calls, validation).
- You want to swap implementations without touching every caller.

The Problem It Solves

Without a factory, every caller has to know about every concrete class:

```
Vehicle v;
if (type.equals("car")) {
    v = new Car();
} else if (type.equals("bike")) {
    v = new Bike();
} else if (type.equals("truck")) {
    v = new Truck();
}
```

Every new vehicle type means editing every call site. A factory pulls this logic into one place.

---

1. Simple Factory

The most common version. Not technically a GoF pattern, but it’s what most people mean by “factory” in casual conversation.

```java
public interface Vehicle {
    void drive();
}

public class Car implements Vehicle {
    public void drive() {
        System.out.println("driving a car");
    }
}

public class Bike implements Vehicle {
    public void drive() {
        System.out.println("riding a bike");
    }
}

public class VehicleFactory {
    public static Vehicle create(String type) {
        switch (type) {
            case "car":
                return new Car();
            case "bike":
                return new Bike();
            default:
                throw new IllegalArgumentException("unknown type: " + type);
        }
    }
}

// Usage
Vehicle v = VehicleFactory.create("car");
v.drive();
```

Pros: dead simple, caller has zero coupling to concrete classes.

Cons: adding a new type still means editing the factory's switch.

---

2. Factory Method

The GoF version. A creator class declares a factory method, and subclasses decide which concrete product to return. Useful when the "which class" decision lives inside a class hierarchy.

```java
public abstract class VehicleShop {
    public abstract Vehicle createVehicle();

    public void deliver() {
        Vehicle v = createVehicle();
        v.drive();
        System.out.println("delivered");
    }
}

public class CarShop extends VehicleShop {
    @Override
    public Vehicle createVehicle() {
        return new Car();
    }
}

public class BikeShop extends VehicleShop {
    @Override
    public Vehicle createVehicle() {
        return new Bike();
    }
}

// Usage
VehicleShop shop = new CarShop();
shop.deliver();
```

The base class `VehicleShop` doesn't know which `Vehicle` subclass it's getting. Each shop type controls that.

Pros: clean separation, easy to plug in new vehicle types by adding new shop subclasses.

Cons: more boilerplate. A new product type often means a new creator subclass.

---

3. Abstract Factory

A factory of factories. Creates families of related objects without coupling the calling code to their concrete classes.

```java
// Product interfaces
public interface Button {
    void render();
}

public interface Checkbox {
    void render();
}

// Concrete products (Mac variants mirror these)
public class WindowsButton implements Button {
    public void render() {
        System.out.println("windows button");
    }
}

public class WindowsCheckbox implements Checkbox {
    public void render() {
        System.out.println("windows checkbox");
    }
}

// MacButton and MacCheckbox follow the same shape

// Abstract factory
public interface UIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

// Concrete factories
public class WindowsFactory implements UIFactory {
    public Button createButton() {
        return new WindowsButton();
    }

    public Checkbox createCheckbox() {
        return new WindowsCheckbox();
    }
}

public class MacFactory implements UIFactory {
    public Button createButton() {
        return new MacButton();
    }

    public Checkbox createCheckbox() {
        return new MacCheckbox();
    }
}

// Usage
UIFactory factory = isMac ? new MacFactory() : new WindowsFactory();
```

```
Button btn = factory.createButton();
Checkbox cb = factory.createCheckbox();
```

Use this when products come in matching sets. A `MacButton` shouldn't pair with a `WindowsCheckbox`. The abstract factory enforces "all from the same family".

Pros: groups related products, prevents mixing incompatible variants.

Cons: heaviest of the three. New product types ripple across every concrete factory.

Real Java Examples

The standard library uses factories everywhere:

Where	Factory method	Returns
`Calendar.getInstance()`	Static factory	Default calendar for the locale
`Integer.valueOf(5)`	Static factory	Cached `Integer` for small values
`Boolean.valueOf("true")`	Static factory	`Boolean.TRUE` or `Boolean.FALSE`
`List.of(1, 2, 3)`	Static factory	Immutable list (Java 9+)
`Optional.of(value)`	Static factory	Wraps a value as `Optional`
`Executors.newFixedThreadPool(4)`	Static factory	Configured `ExecutorService`
`LoggerFactory.getLogger(...)`	Static factory	SLF4J `Logger`
`DriverManager.getConnection(...)`	Static factory	JDBC `Connection`

These are all variations of the same idea: hide construction behind a method that has more freedom than a constructor.

Why Static Factory Methods Beat Constructors

Joshua Bloch makes this case in *Effective Java* (Item 1). Static factory methods give you flexibility a constructor can't.

- **They have a name.** `BigInteger.probablePrime(...)` says what it does. `new BigInteger(...)` doesn't.

- **They can return a cached instance.** `Integer.valueOf(5)` reuses a cached object instead of building a new one.
- **They can return a subtype.** `Collections.unmodifiableList(...)` returns a different class than what you pass in.
- **They can vary the returned type by input.** Useful when behavior changes without exposing the concrete class to the caller.

Downsides:

- A class with only static factories and a private constructor can't be subclassed.
- Static factory methods are harder for new developers to find. Stick to a naming convention: `of`, `valueOf`, `from`, `getInstance`, `newInstance`, `create`.

When to Pick Which

Pattern	Pick when
Simple Factory	One type to decide, small number of variants, easy switch logic
Factory Method	Creation logic lives in a class hierarchy, subclasses pick the concrete type
Abstract Factory	Multiple related products that must stay in matching sets
Static factory method	A constructor is limiting (caching, naming, subtype returns)

Common Gotchas

- A simple factory with a giant switch is the same anti-pattern you started with, just moved one layer down. If new types keep showing up, register them in a `Map<String, Supplier<Vehicle>>` at startup instead.
- Factory methods returning `Object` or a too-broad type push the cast back onto the caller. Return the most specific useful type.
- Abstract factory shines when product families exist. With only one product type, it's overkill, and a simple factory will do.
- Factory creates an object. Builder assembles a complex object step by step. Don't mix them up.
- Avoid mutable static state in factories. Two threads calling the factory at once can corrupt shared fields.

- A factory that hits I/O on every call (database, network, file) needs caching or pooling, otherwise it’s a slow constructor wearing a disguise.